

The Readers and Writers Problem

Sharing a database between many readers and exclusive writers

Definition

The **readers and writers problem** is a synchronization problem in which many processes may safely read shared data at the same time, but a writer must have exclusive access.

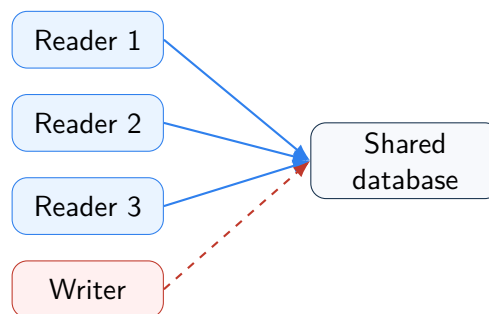
1. Problem Model

Database Access

The problem models systems such as airline reservation databases, where many processes may want to read records and some processes may need to update them.

Access Rule

Multiple readers may access the database simultaneously. If a writer is updating the database, no other process may access it, not even a reader.



2. Core Constraints

Situation	Allowed?
Reader with reader	Yes. Multiple readers can read together.
Writer with reader	No. A writer needs exclusive access.
Writer with writer	No. Only one writer may update at a time.

Main Difficulty

The scheduler must allow reader concurrency without letting readers and writers overlap incorrectly.

3. Semaphores and Counter

Shared Variables

The classic semaphore solution uses two semaphores and a reader count.

Variable	Purpose
<i>mutex</i>	Protects access to the reader count <i>rc</i> .
<i>db</i>	Controls access to the database itself.
<i>rc</i>	Counts how many readers are currently reading or entering.

Why Two Semaphores

mutex protects the bookkeeping variable *rc*. *db* protects the database from writers or from the first reader entering while a writer is active.

4. Reader Algorithm

```
typedef int semaphore;
semaphore mutex = 1; /* controls access to rc */
semaphore db = 1; /* controls access to the database */
int rc = 0; /* number of readers */

void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);

        read_database();

        down(&mutex);
        rc = rc - 1;
        if (rc == 0) up(&db);
        up(&mutex);

        use_data_read();
    }
}
```

First Reader Locks Database

The first reader does a down on *db*. This prevents writers from entering while readers are active.

Last Reader Unlocks Database

The last reader does an up on *db*. This allows a waiting writer to enter after all readers are gone.

5. Writer Algorithm

```
void writer(void)
{
    while (TRUE) {
```

```
    think_up_data();  
    down(&db);  
    write_database();  
    up(&db);  
}  
}
```

Writer Rule

A writer simply waits for exclusive access to *db*, writes the database, and then releases *db*.

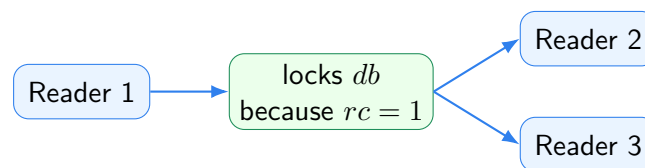
Exclusive Access

While a writer holds *db*, no reader or other writer can access the database.

6. How Readers Enter

Reader Grouping

Once the first reader has locked *db*, later readers do not lock *db* again. They only increment *rc*, read the database, and later decrement *rc*.



Result

Many readers can be inside the database at the same time, but writers are blocked until *rc* becomes 0.

7. Why *mutex* Is Needed

Race on Reader Count

The variable *rc* is shared by all readers. Without *mutex*, two readers could update it at the same time and corrupt the count.

Protected Bookkeeping

Readers hold *mutex* only while changing *rc* and checking whether they are the first or last reader. They do not hold *mutex* while reading the database.

8. Correctness Properties

Property	Reason
Multiple readers allowed	After the first reader locks <i>db</i> , later readers can enter by updating <i>rc</i> .
Writer is exclusive	A writer must hold <i>db</i> , and readers keep <i>db</i> locked while any reader exists.
No writer-writer overlap	The <i>db</i> semaphore allows only one writer to enter.
Reader count is safe	Updates to <i>rc</i> are protected by <i>mutex</i> .

9. Subtle Policy Decision

Reader Preference

The presented solution gives preference to readers. If a reader is active, later readers are allowed to enter even if a writer is already waiting.

Why This Increases Concurrency

Letting new readers join existing readers keeps the database busy and allows many read operations to proceed together.

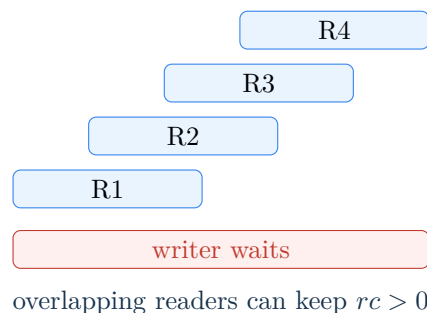
10. Writer Starvation

Starvation Risk

If readers keep arriving while at least one reader is active, a waiting writer may never get access to the database.

Continuous Readers

Suppose a new reader arrives every 2 seconds, and each reader takes 5 seconds to finish. There may always be at least one reader active, so a writer waiting on *db* may never enter.



11. Writer-Priority Alternative

Modified Rule

If a writer is waiting, newly arriving readers can be suspended behind the writer instead of being admitted immediately.

Effect

The writer only waits for readers that were already active when the writer arrived. It does not wait for readers that arrive later.

Tradeoff

Writer priority reduces writer starvation, but it can reduce concurrency because readers may be blocked even though other readers are already reading.

12. Fairness Choices

Policy	Benefit	Cost
Reader preference	High reader concurrency.	Writers may starve.
Writer preference	Writers do not wait behind newly arriving readers.	Reader concurrency can be reduced.
Fair ordering	Reduces starvation for both groups.	More complex to implement.

13. Comparison with Dining Philosophers

Problem	Shared Resource Pattern	Main Lesson
Dining philosophers	Several processes need two exclusive resources.	Avoid deadlock and starvation when acquiring multiple resources.
Readers and writers	Many readers can share, but writers need exclusivity.	Balance concurrency against fairness.

14. Big Picture

Main Conclusion

The readers and writers problem shows how to allow high concurrency for readers while preserving exclusive access for writers. The classic semaphore solution is simple and efficient for readers, but it can starve writers if readers keep arriving.

Exam Shortcut

Remember: *mutex* protects the reader count *rc*, while *db* protects the database. The first reader locks *db*, the last reader unlocks *db*, and writers lock *db* exclusively.

15. Quick Review

Key Points

- The readers and writers problem models shared database access.
- Multiple readers may read at the same time.
- A writer needs exclusive access.
- The semaphore *mutex* protects the reader count *rc*.
- The semaphore *db* protects the database.
- The first reader locks *db*.
- Later readers only increment *rc* and read.
- The last reader unlocks *db*.
- Writers do down on *db*, write, then up on *db*.
- The basic solution gives preference to readers.
- Continuous readers can starve a waiting writer.
- Writer-priority solutions avoid this by blocking new readers behind waiting writers.